

CONTENTS

Entrada / Salida	
Guía detallada desde cero — con los detalles que salvan el parcial	
0. Por qué existe este problema	
1. Buses	
1.1 Valores en los buses durante una operación de E/S	
1.2 Clasificación de buses	
2. Periféricos y controladores de E/S	
2.1 Dónde vive el controlador	
3. Registros de E/S	
4. Máscaras de bits: AND, OR, XOR	
4.1 Leer (consultar) un bit específico sin modificar nada	
5. Polling	
5.1 Dónde se ubica el polling dentro de un sistema mayor	
6. Diseño de un sistema de E/S dedicado: sensor de temperatura + ventilador	
7. Preguntas teóricas frecuentes (con respuesta directa)	
8. Tabla resumen general	
Ejercicios de repaso rápido (con respuesta)	
Resumen ultra-rápido (2 minutos)	

Entrada / Salida

GUÍA DETALLADA DESDE CERO — CON LOS DETALLES QUE SALVAN EL PARCIAL

Arquitectura de Computadoras — FING UdelAR

Buses · Periféricos y controladores · Registros de E/S · Máscaras de bits · Polling · Diseño de un sistema de E/S dedicado

Cómo usar esta guía: cada tema tiene definición corta, procedimiento paso a paso, al menos dos ejemplos resueltos con verificación explícita, tabla resumen donde aplica, y un recuadro rojo de error frecuente. Todo el código en C está verificado bit a bit — cada operación con máscaras (AND/OR/XOR) se comprobó con Python antes de escribirse acá. Este módulo casi siempre aparece combinado con Interrupciones (Módulo 8) en un mismo ejercicio de examen: “sistema embebido con sensores + timer”. Acá se cubre la parte de E/S programada (polling); el mecanismo de interrupciones en sí (INT/INTA, prioridades, ISR) se desarrolla en la guía M8, aunque la pregunta teórica sobre INT/INTA se responde acá porque suele aparecer en el mismo bloque de preguntas teóricas que buses y controladores.

0. Por qué existe este problema

La CPU solo sabe hacer una cosa: ejecutar instrucciones que leen y escriben en memoria y registros. Pero una computadora tiene que interactuar con el mundo real — teclados, sensores, motores, pantallas, válvulas — y ese mundo real no habla en instrucciones de máquina. Necesita **electrónica intermedia** (los controladores de E/S) que traduzca “prender el motor” o “leer el sensor” a algo que la CPU pueda hacer con sus instrucciones normales: leer y escribir en una dirección.

Esa es la idea central de todo el módulo: **un periférico se controla haciendo que sus señales físicas (encendido/apagado, dato leído, alarma activa) se mapeen a bits concretos de un byte al que la CPU accede como si fuera una posición de memoria**. Todo lo que sigue — buses, registros, máscaras, polling — es la mecánica para leer y escribir esos bits sin romper los demás bits del mismo registro, que suelen controlar *otras* cosas del mismo dispositivo.

Idea que hay que tener siempre presente: en un registro de E/S casi nunca todos los bits significan lo mismo. Un mismo byte puede tener el bit 0 controlando un motor, el bit 2 una

válvula, y los bits 3-7 sin usar. Por eso **todo** acceso a E/S se hace con máscaras: nunca se sobrescribe el registro entero salvo que se sepa con certeza el valor de cada bit.

1. Buses

Definición: un bus es un conjunto de líneas físicas que transportan información entre la CPU, la memoria y los dispositivos de E/S. Se distinguen tres buses lógicos (aunque físicamente puedan compartir cableado):

- **Bus de direcciones:** transporta la dirección de memoria o de E/S que se quiere leer/escribir. Es unidireccional (CPU → periféricos/memoria).
- **Bus de datos:** transporta el dato que se transfiere. Es bidireccional (lectura: dispositivo → CPU; escritura: CPU → dispositivo).
- **Bus de control:** transporta las señales que coordinan la transacción: READ/WRITE, IO/M (E/S vs memoria), READY, INT, INTA, reloj de sincronismo, etc.

1.1 VALORES EN LOS BUSES DURANTE UNA OPERACIÓN DE E/S

Esto es exactamente la pregunta (a) típica del práctico: **¿qué se ve en los tres buses al hacer E/S, distinguiendo E/S aislada de E/S mapeada a memoria?**

	E/S Aislada (☐ in / ☐ out)	E/S Mapeada a Memoria
Bus de direcciones	Dirección de E/S (espacio de direcciones separado del de memoria, típicamente más chico)	Dirección de memoria (mismo espacio que usan las instrucciones normales tipo MOV)
Bus de datos	El dato leído/escrito del registro de E/S	Igual — el dato leído/escrito
Bus de control	Señal específica que indica “esto es un acceso de E/S” (ej. IO/M = 1) + READ o WRITE	Señal que indica “esto es un acceso de memoria” (IO/M = 0) + READ o WRITE — no hay señal especial de E/S

Por qué importa la distinción: en E/S aislada, el procesador tiene instrucciones dedicadas (☐ IN, ☐ OUT en x86) y un espacio de direcciones de E/S separado del de memoria — por eso puede haber una dirección de memoria 0x20 y una dirección de **E/S** 0x20 sin conflicto, son espacios distintos. En E/S mapeada a memoria, los registros de los controladores ocupan posiciones del mismo espacio de direcciones que la memoria RAM, y se accede con

las mismas instrucciones (MOV, etc.) — no hace falta instrucción especial, pero se “gasta” espacio de direcciones de memoria.

Error frecuente: decir que en E/S mapeada a memoria “no hay diferencia en el bus de control”. Sí la hay, pero es la **ausencia** de la señal distintiva de E/S — la CPU trata la transacción exactamente igual que un acceso a RAM, y es el **diseño físico del mapa de direcciones** (qué rango de direcciones responde el controlador de E/S vs. la memoria) lo que separa ambos mundos, no una señal de control adicional.

1.2 CLASIFICACIÓN DE BUSES

Según su complejidad:

- **Bus simple (Master–Slave):** una única entidad (típicamente la CPU) es el “master” en todo momento; el resto son “slaves” que esperan ser habilitados.
- **Bus inteligente:** cualquier entidad conectada puede convertirse en “master”; existe un mecanismo de **arbitraje** que decide quién controla el bus en cada transacción.

Según el mecanismo de arbitraje (solo aplica a buses inteligentes):

- **Arbitraje centralizado:** una entidad de mayor jerarquía (típicamente la CPU) entrega el control del bus a quien lo solicite.
- **Arbitraje distribuido:** todas las entidades tienen el mismo rango; cuando hay conflicto se resuelve por un mecanismo tipo “votación” entre las contendientes, sin árbitro central.

Según su aplicación:

- **Interno:** conecta elementos dentro del gabinete de la computadora.
- **Externo:** conecta elementos fuera del gabinete (buses de expansión como PCI Express).

Ejemplo para fijar la clasificación: el bus interno clásico de una PC vieja con un único procesador que siempre decide quién habla es un **bus simple** (Master-Slave) con la CPU como master fijo. Un bus tipo PCI moderno, donde varias tarjetas pueden iniciar transferencias (DMA) y hay un árbitro que asigna el bus, es un **bus inteligente con arbitraje centralizado**.

2. Periféricos y controladores de E/S

Periférico: dispositivo que permite la interacción del sistema con el mundo exterior — transductores (teclado, mouse, impresora, monitor, escáner), almacenamiento (disco, cinta), comunicación (módem, red).

Controlador de E/S: es la parte “inteligente” del periférico — el circuito que efectivamente lo controla y que expone registros accesibles por la CPU.

2.1 DÓNDE VIVE EL CONTROLADOR

El controlador puede residir en distintos lugares, y de eso depende qué interfaz o adaptador hace falta:

1. **El controlador está en el gabinete del computador:**
 - (a) conectado directamente al bus interno del sistema, o
 - (b) conectado a un bus distinto, que se une al bus interno mediante un **adaptador de bus**.
2. **El controlador está en el gabinete del periférico** (fuera de la computadora): no puede conectarse directamente al bus interno, así que se necesita una **interfaz** (protocolo simple, para un único dispositivo):
 - (a) la interfaz se conecta directamente al bus interno,
 - (b) se usa un adaptador de bus conectado directamente al bus interno,
 - (c) se usa un adaptador de bus conectado a un bus distinto del interno.
3. **El controlador está distribuido** entre el gabinete del computador y el del periférico (parte de la lógica en cada lado).

Regla práctica para el examen: si te preguntan “¿qué necesita este periférico para conectarse?”, pensá primero *dónde físicamente está el controlador* (adentro o afuera de la torre) y después qué bus usa — de ahí sale directo si hace falta interfaz, adaptador, o ambos.

3. Registros de E/S

Definición: un controlador de E/S expone un conjunto pequeño de “posiciones de memoria especiales” llamadas registros, mediante las cuales la CPU intercambia información con él. Hay cuatro comportamientos posibles de acceso:

Tipo	Comportamiento	Ejemplo típico
Solo lectura	Se puede leer; si se escribe, no tiene ningún efecto.	<code>ESTADO_TECLADO</code> : bit indica si hay una tecla disponible.
Solo escritura	Se puede escribir; si se lee, el resultado es impredecible.	<code>DISPLAY</code> de 7 segmentos: se le escribe el dígito a mostrar, no tiene sentido leerlo.
Lectura/escritura independiente	Dos registros <i>distintos</i> en la misma dirección de E/S: uno solo-lectura y otro solo-escritura. Lo que se escribe no se puede releer después — son dos posiciones de memoria físicamente distintas que comparten dirección.	Un puerto serie clásico: escribir en la dirección envía un byte por la línea; leer la misma dirección devuelve el próximo byte recibido — son dos buffers de hardware distintos.
Lectura/escritura normal	Se comporta como una posición de memoria común: lo que se escribe se puede leer después, y es el mismo dato físico.	Un registro de configuración simple: se escribe un modo y luego se puede releer para confirmarlo.

Ejemplo 1 (Solo lectura): en el Ejercicio 1 del Práctico 9, `ESTADO_TECLADO` y `DATO_TECLADO` son de solo lectura: el hardware los actualiza solo, y si el programa intentara hacer `out(DATO_TECLADO, x)` no pasaría nada.

Ejemplo 2 (Lectura/escritura normal — distinto de “independiente”): en el Ejercicio 2 del Práctico 9, el puerto `BOMBA` es un registro **normal**: la CPU lee su valor actual (para saber qué bits ya estaban prendidos, como el del sensor), modifica solo el bit que le interesa con una máscara, y vuelve a escribir el registro completo. Esto **solo funciona** porque es lectura/escritura normal: si fuera lectura/escritura independiente, leer `BOMBA` devolvería el estado real del hardware (sensor, etc.) pero **no** lo último que la CPU escribió — y el patrón “leo, modifico con máscara, escribo” dejaría de tener sentido para los bits de salida.

Error frecuente de examen: confundir “lectura/escritura independiente” con “solo lectura + solo escritura por separado en direcciones distintas”. **No es lo mismo**. En lectura/escritura independiente ambos registros comparten la **misma dirección de E/S** — es la operación (IN vs OUT) la que determina a cuál de los dos registros físicos se accede, no la dirección.

4. Máscaras de bits: AND, OR, XOR

Por qué hacen falta: como un registro de E/S casi nunca tiene todos sus bits con el mismo significado, **nunca** se puede escribir un valor completo nuevo sin antes saber (o preservar) el estado de los demás bits. Las máscaras permiten operar sobre **un subconjunto de bits sin tocar el resto**.

Las tres operaciones:

- **AND con máscara de ceros en las posiciones a limpiar (y unos en las que se preservan):** sirve para **apagar (poner en 0)** bits específicos, dejando el resto intacto.
- **OR con máscara de unos en las posiciones a setear (y ceros en las que se preservan):** sirve para **prender (poner en 1)** bits específicos, dejando el resto intacto.
- **XOR con máscara de unos en las posiciones a invertir:** sirve para **invertir (toggle)** bits específicos, dejando el resto intacto.

Procedimiento general para modificar un bit de un registro de E/S:

1. Leer el registro completo: `char reg = in(DIRECCION);`
2. Aplicar la máscara con la operación correspondiente: `reg = reg OP 0xMASCARA;`
3. Escribir el registro completo de vuelta: `out(DIRECCION, reg);`

Ejemplo 1 — prender el bit 2 (OR) sin tocar el resto.

Registro actual: `0000 0000` (0x00). Máscara para bit 2: `0000 0100` (0x04).

`0x00 | 0x04 = 0x04` → `0000 0100`.

Verificación bit a bit: bit2: 0 OR 1 = 1 ✓. Resto de bits: x OR 0 = x (sin cambio) ✓.

Ejemplo 2 — apagar el bit 4 (AND) sin tocar el resto.

Registro actual: `0001 0101` (0x15, bit4=1, bit2=1, bit0=1). Máscara para “apagar bit4 y preservar todo lo demás”: `1110 1111` (0xEF — es el complemento de 0x10).

`0x15 & 0xEF = 0x05` → `0000 0101`.

Verificación bit a bit: bit4: 1 AND 0 = 0 ✓ (apagado). bit2: 1 AND 1 = 1 ✓ (preservado). bit0: 1 AND 1 = 1 ✓ (preservado).

Truco para construir la máscara de AND (apagar bits): la máscara siempre es el **complemento** (NOT) de la máscara que usarías con OR para prender esos mismos bits. Si para prender el bit N usás `0x_M`, para apagarlo usás `~0x_M` (en 8 bits, `0xFF - 0x_M`). Ejemplo: prender bit4 → 0x10; apagar bit4 → 0xFF-0x10=0xEF. Prender bit0 → 0x01; apagar bit0 → 0xFE. Prender bit2 → 0x04; apagar bit2 → 0xFB.

Ejemplo 3 — invertir (toggle) el bit 0 con XOR.

Registro actual: `0000 0001` (`0x01`). Máscara: `0x01`.

`0x01 ^ 0x01 = 0x00`. Aplicando XOR una segunda vez: `0x00 ^ 0x01 = 0x01`.

Verificación: XOR con 1 invierte el bit ($0 \rightarrow 1$ o $1 \rightarrow 0$); XOR con 0 lo deja igual — por eso la máscara XOR tiene 1 solo en los bits a invertir y 0 en el resto.

4.1 LEER (CONSULTAR) UN BIT ESPECÍFICO SIN MODIFICAR NADA

Para **consultar** el valor de un bit (no para escribirlo), se usa AND con máscara de un solo 1 en la posición de interés, y se compara el resultado contra 0:

```
if ((in(ESTADO) & 0x80) != 0) {
    // el bit 7 está en 1
}
```

Error de tolerancia cero — **precedencia de operadores en C**: el operador `!=` tiene **mayor precedencia** que `&`. Esto significa que `in(ESTADO) & 0x80 != 0` (sin paréntesis) se interpreta como `in(ESTADO) & (0x80 != 0)`, y como `0x80 != 0` es cierto (vale 1 lógico), la expresión completa queda `in(ESTADO) & 1` — que consulta el **bit 0**, no el bit 7. Esta es una solución real de un práctico de la materia con este exacto error. **Regla de tolerancia cero: siempre poner paréntesis explícitos alrededor del AND/OR/XOR:**

`(in(ESTADO) & 0x80) != 0`.

Verificación con Python de la comparación errónea (para convencerte): `0x01 != 1` en C vale `0` (falso, porque son iguales). Entonces `x & 0x01 != 1` se vuelve `x & 0`, que **siempre da 0** — un bucle de espera con esa condición nunca se cumple o nunca termina, según cómo esté escrito. Comprobado: `python -c "print(int(1!=1))"` $\rightarrow$ `0`; `python -c "print(0x05 & 0)"` $\rightarrow$ `0`.

5. Polling

Definición: técnica de espera activa en la que el programa consulta repetidamente (en un bucle) un bit de estado hasta que indique que el dispositivo está listo. Es fácil de implementar (no requiere hardware ni software adicional para gestionar eventos), pero **ineficiente**: la CPU no puede hacer nada más mientras espera.

Procedimiento general:

1. Leer el registro de estado.
2. Aplicar la máscara del bit de interés y comparar.
3. Si la condición de “no listo” se cumple, repetir el bucle (volver al paso 1).
4. Cuando el bit indica “listo”, salir del bucle y leer/escribir el dato.

Ejemplo 1 — leer un carácter del teclado por polling (corrección del Ejercicio 1, Práctico 9).

Enunciado: `ESTADO_TECLADO` y `DATO_TECLADO` son registros de un byte, de solo lectura. El hardware pone el bit más significativo (bit 7) de `ESTADO_TECLADO` en 1 cuando hay un carácter disponible, y en 0 automáticamente al leer `DATO_TECLADO`.

```
#define MAX_CHAR 1024

char buffer[MAX_CHAR];
int tope = 0;

void leerTeclado() {
    while (1) {
        // Mientras el bit 7 de ESTADO_TECLADO esté en 0, no hay dato: sigo es-
        perando.
        while ((in(ESTADO_TECLADO) & 0x80) == 0)
            ; // espera activa

        // Bit 7 en 1: hay un caracter disponible.
        buffer[tope] = in(DATO_TECLADO); // leerlo apaga el bit 7 automáticamente
        (lo hace el hardware)
        tope = (tope + 1) % MAX_CHAR;      // lista circular de largo variable
    }
}
```

Verificación de la condición: si `ESTADO_TECLADO = 0x80` (hay dato), `0x80 & 0x80 = 0x80`, que es `!= 0`, entonces `== 0` es falso → el `while` interno termina, se lee el dato. Si `ESTADO_TECLADO = 0x00` (no hay dato), `0x00 & 0x80 = 0x00`, que `== 0` es verdadero → se sigue esperando. Correcto en ambos casos.

Discrepancia encontrada con la solución oficial del Práctico 9 (PDF “Ejercicios Resueltos”, Ejercicio 1): el código allí escribe `while ((in(ESTADO_TECLADO) & 0x80) == 1);`. Esto es un error: `in(ESTADO_TECLADO) & 0x80` solo puede dar `0x00` o `0x80` (128), **nunca** el valor `1` — comparar contra `1` hace que la condición sea **siempre falsa**, y el bucle de espera nunca esperaría (seguiría de largo incluso sin dato disponible, leyendo basura). La versión correcta, usada arriba, es comparar contra `0`: `(in(ESTADO_TECLADO) &`

`0x80) == 0` para “esperar mientras NO hay dato”. Verificado bit a bit arriba y con `python -c "print(hex(0x80 & 0x80))"` → `0x80` (no `1`).

Ejemplo 2 — activar/desactivar la bomba de agua (Ejercicio 2, Práctico 9) usando polling sobre el sensor.

Enunciado: puerto `BOMBA` (lectura/escritura normal, 1 byte). Bit 2 = orden de encendido/apagado (CPU → bomba). Bit 0 = sensor externo, indica si la bomba está realmente activa. Bit 4 = LED que refleja el estado real (bit 0).

```
void activarBomba() {
    char reg = in(BOMBA);
    reg = reg | 0x04;          // bit2=1: ordeno encender
    out(BOMBA, reg);

    // Polling: espero a que el sensor (bit0) confirme que ya arrancó.
    while ((in(BOMBA) & 0x01) == 0)
        ;

    reg = in(BOMBA);
    reg = reg | 0x10;          // bit4=1: prendo el LED de confirmación
    out(BOMBA, reg);
}

void desactivarBomba() {
    char reg = in(BOMBA);
    reg = reg & 0xFB;          // bit2=0: ordeno apagar (0xFB = ~0x04)
    out(BOMBA, reg);

    // Polling: espero a que el sensor (bit0) confirme que ya se detuvo.
    while ((in(BOMBA) & 0x01) != 0)
        ;

    reg = in(BOMBA);
    reg = reg & 0xEF;          // bit4=0: apago el LED (0xEF = ~0x10)
    out(BOMBA, reg);
}
```

Verificación: `0xFB` = complemento de `0x04` (`0000 0100` invertido = `1111 1011` = `0xFB`)
 ✓. `0xEF` = complemento de `0x10` (`0001 0000` invertido = `1110 1111` = `0xEF`) ✓, ambos comprobados con `python -c "print(hex((~0x04)&0xFF))"` → `0xfb` y `python -c "print(hex((~0x10)&0xFF))"` → `0xef`.

Discrepancia adicional encontrada en la solución oficial del Ejercicio 2: el PDF escribe las condiciones de espera como `while (in(BOMBA) & 0x01 != 1);` y `while (in(BOMBA) & 0x01 == 1);`, ****sin paréntesis**** alrededor del AND. Por la regla de precedencia de la sección 4.1 (`!=`/`==` se evalúan antes que `&`), estas expresiones no hacen lo que el enunciado necesita. Se corrigió arriba agregando paréntesis explícitos y ajustando qué valor de comparación corresponde a “esperando” en cada caso (`bit0=0` = bomba parada, `bit0=1` = bomba activa).

5.1 DÓNDE SE UBICA EL POLLING DENTRO DE UN SISTEMA MAYOR

Cuando un controlador **no tiene capacidad de generar interrupciones**, hay dos estrategias (según el resumen de la materia):

- **Polling en un timer:** un reloj interrumpe periódicamente y en su rutina se hace la consulta al controlador. Tiene un retardo de reacción igual al período del timer.
- **Polling en el programa principal:** el bucle principal consulta directamente. Reacciona instantáneamente, pero bloquea cualquier otra tarea mientras espera.

6. Diseño de un sistema de E/S dedicado: sensor de temperatura + ventilador

Enunciado (elegido para ilustrar el flujo completo): un sistema embebido, dedicado exclusivamente a esta tarea, debe leer continuamente la temperatura de un sensor y encender un ventilador cuando la temperatura supere un umbral, apagándolo cuando vuelva a bajar. Además debe encender un LED de alarma mientras el ventilador esté activo.

Registros:

- `TEMPERATURA` — dirección de E/S, **solo lectura**, 1 byte. Contiene la temperatura actual en grados Celsius como entero sin signo (0–255), actualizada por el hardware del sensor.
- `CONTROL_VENTILADOR` — dirección de E/S, **lectura/escritura normal**, 1 byte. Bit 0 = ventilador (1=encendido, 0=apagado). Bit 1 = LED de alarma (1=encendido, 0=apagado). Bits 2–7 reservados (no usar).

Constante: `UMBRAL = 30` grados.

Máscaras necesarias (todas verificadas en la sección 4): prender ventilador → OR con `0x01`; apagar ventilador → AND con `0xFE`; prender LED → OR con `0x02`; apagar LED → AND con `0xFD`; consultar si el ventilador está prendido → AND con `0x01` y comparar `!= 0`.

```

#define UMBRAL 30
#define VENTILADOR_BIT 0x01
#define LED_ALARMA_BIT 0x02

// Máquina dedicada: el procesador solo hace esto, así que el "polling"
// del sensor se hace directamente en el programa principal (reacción
// instantánea, sin depender de un timer).
void main() {
    while (1) {
        char temp = in(TEMPERATURA);
        char ctrl = in(CONTROL_VENTILADOR);
        int ventiladorPrendido = (ctrl & VENTILADOR_BIT) != 0;

        if (temp > UMBRAL && !ventiladorPrendido) {
            // Supera el umbral y el ventilador estaba apagado: prender todo.
            ctrl = ctrl | VENTILADOR_BIT;
            ctrl = ctrl | LED_ALARMA_BIT;
            out(CONTROL_VENTILADOR, ctrl);
        }
        else if (temp <= UMBRAL && ventiladorPrendido) {
            // Bajó del umbral y el ventilador estaba prendido: apagar todo.
            ctrl = ctrl & (char)(~VENTILADOR_BIT);
            ctrl = ctrl & (char)(~LED_ALARMA_BIT);
            out(CONTROL_VENTILADOR, ctrl);
        }
        // Si no cambió nada respecto al estado actual, no se toca el registro
        // (evita escrituras innecesarias – buena práctica, no solo estilo).
    }
}

```

Verificación bit a bit (traza completa):

1. Estado inicial: `CONTROL_VENTILADOR = 0x00` (todo apagado). `TEMPERATURA = 35` ($^{\circ} > 30$).
2. `ctrl & 0x01 = 0x00 & 0x01 = 0` → `ventiladorPrendido = 0` (falso).
3. `temp > UMBRAL` ($35 > 30$, verdadero) y `!ventiladorPrendido` (verdadero) → entra al primer `if`.
4. `ctrl = 0x00 | 0x01 = 0x01`. `ctrl = 0x01 | 0x02 = 0x03`. Se escribe `CONTROL_VENTILADOR = 0x03` (`0000 0011`: bit0=1 ventilador, bit1=1 LED) ✓.
5. Pasa el tiempo, `TEMPERATURA` baja a `25` (≤ 30). `ctrl` leído ahora es `0x03`.
6. `ctrl & 0x01 = 0x03 & 0x01 = 0x01 ≠ 0` → `ventiladorPrendido = 1` (verdadero).
7. `temp <= UMBRAL` (verdadero) y `ventiladorPrendido` (verdadero) → entra al segundo `if`.
8. `ctrl = 0x03 & (~0x01 & 0xFF) = 0x03 & 0xFE = 0x02`. `ctrl = 0x02 & (~0x02 & 0xFF) = 0x02 & 0xFD = 0x00`. Se escribe `CONTROL_VENTILADOR = 0x00` (todo apagado) ✓.

Todas las operaciones de esta traza fueron comprobadas con Python: `hex(0x00|0x01) → 0x1`, `hex(0x01|0x02) → 0x3`, `hex(0x03&0xfe) → 0x2`, `hex(0x02&0xfd) → 0x0`.

Chequeo de diseño (no solo de bits): notá que el código nunca hace

`out(CONTROL_VENTILADOR, 0x03)` directamente con una constante fija — siempre lee, aplica máscara sobre el valor leído, y escribe. Esto es **necesario** incluso en este ejemplo simple porque hay dos bits relacionados (ventilador y LED) en el mismo registro: escribir una constante fija sin leer antes solo es seguro si estás 100% seguro de que ya conocés el valor de **todos** los bits del registro en ese instante — y en un sistema real eso rara vez es cierto sin leer primero.

Error de diseño frecuente: usar un `if` sin la condición negada (`!ventiladorPrendido` / chequeo de que ya estaba prendido) y volver a escribir el registro en cada vuelta del bucle aunque no haya cambiado nada. Esto no rompe la lógica pero es mala práctica de E/S: cada `out()` es una operación de bus real; escribir sin necesidad puede interferir con otros bits que otro proceso (o interrupción) esté modificando concurrentemente, o simplemente desperdicia ancho de banda del bus.

7. Preguntas teóricas frecuentes (con respuesta directa)

(b) Propósito del controlador de interrupciones: actúa como intermediario entre múltiples controladores de E/S y la única entrada `INT` de la CPU. Cuando algún controlador solicita una interrupción, el controlador de interrupciones genera el pedido hacia la CPU y, cuando la CPU responde con `INTA`, es **el controlador de interrupciones** (no el dispositivo directamente) quien coloca en el bus de datos el identificador correspondiente a la línea `IRQ` de origen. Centraliza así la gestión de prioridades entre múltiples fuentes de interrupción, algo que la CPU no podría resolver sola con una única línea `INT`.

(c) Mecanismo INT/INTA y Daisy Chain: con una única entrada `INT` en la CPU conectada en OR cableado a todos los controladores, la CPU no sabe por sí sola quién generó el pedido. Al aceptar la interrupción, la CPU sube la señal `INTA` (“interrupt acknowledge”). En una conexión **Daisy Chain**, la señal `INTA` se propaga en cadena, dispositivo por dispositivo, físicamente ordenados: el primero que tenga un pedido pendiente “atrapa” la señal `INTA` y no la deja pasar al siguiente, identificándose así ante la CPU. **Sí existe una prioridad implícita:** el dispositivo conectado más cerca del inicio de la cadena (el que recibe primero la señal `INTA`) tiene prioridad más alta, porque intercepta la señal antes de que le llegue a los siguientes.

(d) Máquina dedicada vs. no dedicada:

- **Dedicada:** el procesador ejecuta un único sistema, desarrollado por un mismo equipo, sin otros programas concurrentes. Todas las rutinas (principal e interrupciones) pueden coordinar el uso de recursos compartidos por convención de diseño, sin necesidad de proteger el contexto de “otros” programas porque no los hay.
- **No dedicada:** coexisten múltiples programas de propósito distinto. Una rutina de interrupción **no puede** asumir nada sobre qué estaba haciendo el programa interrumpido, y debe preservar explícitamente todo registro que use (guardarlo al entrar, restaurarlo al salir) para no corromper el contexto de un programa que no tiene relación con ella.

Consecuencia práctica sobre el diseño: en una máquina dedicada el polling puede hacerse tranquilamente en el `while(1)` del programa principal (como en la sección 6) porque no hay “otra cosa” que la CPU debería estar haciendo mientras tanto. En una máquina no dedicada, ese mismo polling bloquearía injustamente a todos los demás programas — ahí es donde tiene sentido preferir interrupciones (Módulo 8) en vez de polling.

8. Tabla resumen general

Concepto	Idea clave
Bus de direcciones	Selecciona la posición (memoria o E/S) a acceder
Bus de datos	Transporta el valor leído/escrito
Bus de control	Coordina la transacción (R/W, IO/M, INT, INTA, reloj)
E/S aislada	Espacio de direcciones propio; instrucciones <code>IN</code> / <code>OUT</code> dedicadas
E/S mapeada a memoria	Comparte espacio de direcciones con la RAM; se usa <code>MOV</code> normal
Registro solo lectura	Escribir no tiene efecto
Registro solo escritura	Leer da un valor impredecible
Lectura/escritura independiente	Dos registros físicos distintos comparten una dirección; lo escrito no se puede releer
Lectura/escritura normal	Se comporta como memoria común: lo escrito se puede releer igual
Máscara AND	Apaga bits específicos (0 en la posición, 1 en el resto)
Máscara OR	Prende bits específicos (1 en la posición, 0 en el resto)
Máscara XOR	Invierte bits específicos (1 en la posición, 0 en el resto)
Polling	Espera activa consultando un bit de estado en bucle — simple pero ineficiente

Ejercicios de repaso rápido (con respuesta)

Pregunta	Respuesta
Registro de 8 bits <code>0x00</code> , ¿cómo prendo el bit 3 sin tocar el resto?	<code>reg = reg 0x08;</code>
Registro <code>0x0F</code> (<code>0000 1111</code>), ¿cómo apago el bit 1 sin tocar el resto?	<code>reg = reg & 0xFD;</code> → resultado <code>0x0D</code> (<code>0000 1101</code>)
¿Cómo consulto si el bit 5 de <code>ESTADO</code> está en 1, con precedencia correcta?	<code>if ((in(ESTADO) & 0x20) != 0)</code>
<code>in(BOMBA) & 0x01 != 1</code> en C, ¿qué hace realmente?	Se evalúa como <code>in(BOMBA) & (0x01 != 1) = in(BOMBA) & 0</code> = siempre <code>0</code> (bug de precedencia)
¿Qué diferencia hay entre E/S aislada y mapeada a memoria en el bus de direcciones?	Aislada usa un espacio de direcciones de E/S separado; mapeada a memoria usa el mismo espacio que la RAM
En Daisy Chain, ¿quién tiene mayor prioridad?	El dispositivo más cercano al inicio de la cadena (intercepta primero la señal INTA)
¿Qué tipo de registro son <code>ESTADO_TECLADO</code> y <code>DATO_TECLADO</code> del Ejercicio 1?	Solo lectura
¿Por qué el puerto <code>BOMBA</code> debe ser lectura/escritura normal y no independiente?	Porque el código necesita leer el último valor escrito (para preservar otros bits) antes de modificar un bit específico
¿Qué hace <code>reg ^ 0x04</code> dos veces seguidas sobre el mismo registro?	Vuelve al valor original (XOR es su propia inversa)
En una máquina no dedicada, ¿qué debe hacer una rutina de interrupción antes de usar un registro de la CPU?	Guardar su valor actual (para restaurarlo al salir), porque no puede asumir nada del contexto del programa interrumpido

Resumen ultra-rápido (2 minutos)

- **Buses:** direcciones (selecciona), datos (transporta), control (coordina). Simple (1 master fijo) vs. inteligente (arbitraje centralizado o distribuido); interno vs. externo.
- **E/S aislada:** direcciones de E/S propias, `IN` / `OUT`. **E/S mapeada a memoria:** mismo espacio que la RAM, `MOV` normal.

- **Controlador de E/S:** puede estar en el gabinete del computador, del periférico (necesita interfaz), o distribuido.
- **4 tipos de registro:** solo lectura, solo escritura, lectura/escritura independiente (dos registros físicos, misma dirección, no se relee lo escrito), lectura/escritura normal (se relee lo escrito).
- **Máscaras:** OR para prender bits, AND (con el complemento) para apagarlos, XOR para invertirlos. Siempre: leer → aplicar máscara → escribir. Nunca comparar sin paréntesis alrededor del AND (la precedencia de `==` / `!=` en C es mayor).
- **Polling:** `while ((in(ESTADO) & BIT) == 0);` para esperar mientras el bit está en 0. Simple, ineficiente, bloquea la CPU.
- **Diseño dedicado:** en un `while(1)` del `main`, leer sensor y estado, aplicar la lógica de umbral, escribir con máscaras solo si cambia algo.
- **INT/INTA + Daisy Chain:** OR cableado a una sola entrada `INT`; `INTA` se propaga en cadena; el primero en la cadena tiene mayor prioridad.
- **Dedicada vs. no dedicada:** dedicada = un solo sistema, sin necesidad de proteger contexto ajeno; no dedicada = múltiples programas, toda rutina de interrupción debe salvar y restaurar lo que usa.